

This is my SaaS product vision and business workflow.

STRICT RULES:

1. Do NOT change architecture
2. Do NOT merge modules
3. Follow modular, pluggable design
4. Respect tenant isolation and RBAC
5. Follow workflows exactly as defined

Your job is to implement, not redesign.

Engineering Rules:

1. All APIs must be:

- idempotent
- paginated
- optimized (no N+1)

2. Security:

- enforce RBAC
- validate all inputs
- no raw SQL
- protect PII

3. Observability:

- log execution time
- log errors
- track key metrics

4. Performance:

- support concurrent users
- avoid blocking operations

Do not violate these.

Based on the documents:

Task: Design Django models for CRM module

Constraints:

- Must support configurable stages
- Must support conversion → member (via event, not direct dependency)
- Must store lost reasons
- Must be tenant-aware

Give:

1. Models
2. Relationships
3. Key fields
4. Constraints
5. Events emitted

Ask:

Check:

- Any tight coupling?
- Any violation of module independence?
- Any missing dependency?

Phase 1 (Start Here)

 CRM (complete properly)

- Lead model
- Stage config
- Conversion logic
- Lost reasons
- CSV import

Phase 2

 Intake module

Phase 3

👉 Membership + lifecycle integration (refine)

🔥 Phase 4

👉 Booking fixes (capacity, waitlist)

🔥 Phase 5

👉 Attendance validation layer

🔥 Phase 6

👉 Payments

🔥 Phase 7

👉 Reporting

When Claude gives code, ALWAYS ask:

Explain how this module can work independently without other modules.

If it fails → reject that design

PROJECT CONTEXT:

I am building a modular, multi-tenant SaaS platform (ANJASI Ecosystem).

Architecture Principles:

- Modular and pluggable modules (CRM, Members, Payments, Booking, Attendance, etc.)
- No tight coupling between modules
- Modules communicate via events, not direct dependencies
- Multi-tenant architecture (strict tenant isolation)
- RBAC enforced via centralized PermissionService
- Service-layer business logic (no logic in views/controllers)

You MUST NOT:

- redesign architecture
- merge modules
- duplicate existing modules (payments, members, booking)
- bypass RBAC or tenant isolation

ENGINEERING RULES (NON-NEGOTIABLE):

1. PERFORMANCE & SCALABILITY

- All APIs must support pagination
- Avoid N+1 queries (use `select_related` / `prefetch_related`)
- Use efficient queries only (no unnecessary DB hits)
- APIs must be safe for concurrent usage
- Design for at least 500 concurrent users

2. SECURITY (MANDATORY)

- Validate all inputs (no blind trust)
- Use ORM only (no raw SQL unless justified)
- Enforce RBAC at service layer (not just UI)
- Prevent unauthorized access to any resource

- Add rate limiting hooks for sensitive APIs

3. OBSERVABILITY

- Log all API requests (with execution time)
- Log errors with proper context
- Track key operations (create/update/delete)
- Design for future monitoring integration

4. MODULARITY

- Each module must work independently
- No direct cross-module DB access unless explicitly allowed
- Use events for cross-module communication
- Keep module boundaries clean

5. DATA INTEGRITY

- Use transactions for critical operations
- Enforce constraints at DB level
- Prevent duplicate records where applicable

TASK:

Based on the above rules and architecture, implement:

[REPLACE THIS WITH YOUR TASK]

(e.g., "Design CRM models", "Create booking APIs", etc.)

OUTPUT FORMAT:

1. Models (if applicable)
2. Relationships
3. Service layer logic
4. API endpoints
5. Validation rules
6. Events emitted/consumed

7. Performance considerations

8. Security considerations

VALIDATION:

Before finalizing, verify:

- No tight coupling introduced
- Tenant isolation is preserved
- RBAC is enforced
- Performance risks are minimized
- No security vulnerabilities introduced

PII PROTECTION (MANDATORY):

- All sensitive fields (phone, email, health data) must be encrypted at rest
- Default API response must return masked values
- Full data access must go through PII service layer only
- Direct DB access to sensitive data is strictly prohibited
- Every access to sensitive data must be logged (PIIAccessLog)

RBAC EXTENSION:

- Enforce pii.view_masked and pii.view_full permissions

AUDIT REQUIREMENTS:

- Log who accessed what data, when, and from where
- Logs must be immutable

Never expose raw sensitive data without authorization checks.

TERMINOLOGY & ABSTRACTION RULE (NON-NEGOTIABLE):

The system must follow a strict separation between:

1. Backend (Canonical Model)
2. UI (Industry-Specific Vocabulary)

BACKEND RULES:

- Use generic, reusable terminology only
- Do NOT use industry-specific names in models, fields, or APIs
- Examples:
 - Use "Member", NOT "Student" or "Resident"
 - Use "Session", NOT "YogaClass" or "Lecture"
 - Use "Plan", NOT "Course" or "SubscriptionPlan"

UI RULES:

- UI labels must be configurable per tenant/industry
- Same backend entity should appear with different names based on context
- Example:
 - "Member" → "Student" (Coaching)
 - "Member" → "Resident" (Apartment)
 - "Member" → "Client" (Wellness)

IMPLEMENTATION REQUIREMENTS:

- Introduce a "Terminology Mapping Layer"
- Store UI labels in configuration (DB or config file)
- APIs must return canonical field names only
- UI layer is responsible for mapping labels

STRICTLY PROHIBITED:

- Creating duplicate models for different industries
- Hardcoding industry-specific terms in backend
- Mixing UI terminology with database schema

VALIDATION:

Before finalizing any design, confirm:

- Is backend terminology generic?
- Can this module work across multiple industries without change?